

Apostila 5 — Projetos Práticos de Economia

Esta apostila reúne projetos completos que integram tudo que você aprendeu. Cada projeto é independente e usa dados reais.

Projeto 1 — Dashboard Macroeconômico do Brasil

Objetivo

Criar um script que baixa, analisa e gera um relatório completo com os principais indicadores.

Indicadores

Variável	Código BCB
IPCA (mensal)	433
SELIC (meta)	11
Desemprego	24369
Câmbio (USD)	1
PIB mensal	4380

Script completo

```
from bcb import sgs
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

# 1. BAIXAR DADOS
codigos = {
    "IPCA": 433,
    "SELIC": 11,
    "DESEMPREGO": 24369,
    "CAMBIO": 1,
    "PIB": 4380
}

dados = sgs.get(codigos, start="2010-01-01")

# 2. PROCESSAR
# Inflação acumulada 12 meses
dados["IPCA_ACUM"] = dados["IPCA"].rolling(12).sum()

# Juro real
dados["JURO_REAL"] = dados["SELIC"] - dados["IPCA_ACUM"]

# Variação do câmbio
dados["CAMBIO_VAR"] = dados["CAMBIO"].pct_change() * 100

# 3. RELATÓRIO TEXTO
print("=" * 60)
print("RELATÓRIO MACROECONÔMICO - BRASIL")
print("=" * 60)

ultimos = dados.tail(1)
print(f"\n📅 Referência: {ultimos.index[0].date()}")
print(f"\n📈 INFLAÇÃO")
```

```

print(f"    IPCA mensal: {ultimos['IPCA'].values[0]:.2f}%")
print(f"    IPCA acum. 12m: {ultimos['IPCA_ACUM'].values[0]:.2f}%")

print(f"\n 📌 JUROS")
print(f"    SELIC: {ultimos['SELIC'].values[0]:.2f}%")
print(f"    Juro real: {ultimos['JURO_REAL'].values[0]:.2f}%")

print(f"\n 📌 MERCADO DE TRABALHO")
print(f"    Desemprego: {ultimos['DESEMPREGO'].values[0]:.2f}%")

print(f"\n 📌 CÂMBIO")
print(f"    USD/BRL: {ultimos['CAMBIO'].values[0]:.2f}")

# 4. GRÁFICOS
fig, axes = plt.subplots(3, 2, figsize=(14, 10))
axes = axes.flatten()

titulos = [
    ("IPCA (mensal)", "crimson"),
    ("SELIC", "navy"),
    ("IPCA Acum. 12m", "darkred"),
    ("Desemprego", "darkgreen"),
    ("Câmbio (USD/BRL)", "purple"),
    ("Juro Real", "orange")
]

for ax, (titulo, cor), (_, serie) in zip(axes, titulos, dados[titulos]):
    ax.plot(dados.index, serie, color=cor, linewidth=1.2)
    ax.set_title(titulo, fontsize=12, fontweight="bold")
    ax.grid(True, alpha=0.3)

fig.delaxes(axes[5]) # remove último (vazio)
fig.tight_layout()
fig.savefig("dashboard_macro.png", dpi=200, bbox_inches="tight")
plt.close()

print(f"\n 📄 Dashboard salvo: dashboard_macro.png")

```

Desafio

Adicione mais 2 indicadores de sua escolha (pesquise códigos no site do BCB SGS).

Projeto 2 — Análise de Convergência de Renda

Objetivo

Testar a hipótese de β -convergência: países pobres crescem mais rápido que países ricos.

Dados

Vamos usar o World Development Indicators (WDI) do Banco Mundial.

```

import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
from pandas_datareader import wb # alternativa: baixar CSV manualmente

```

```

# Baixar PIB per capita (método alternativo)
# NY.GDP.PCAP.PP.KD = PIB per capita PPP (US$ constantes)
países = ["BRA", "ARG", "CHL", "COL", "PER", "MEX", "URY", "ECU",
          "USA", "CAN", "GBR", "DEU", "FRA", "JPN", "KOR", "CHN",
          "IND", "ZAF", "NGA", "KEN", "ETH", "VNM", "IDN", "PHL",
          "TUR", "RUS", "SAU", "ARE"]

# NOTA: Se pandas_datareader não estiver instalado:
# pip install pandas_datareader
# Se falhar, baixe manualmente de https://data.worldbank.org/

# Vou usar dados simulados para demonstração:
np.random.seed(42)
anos_dados = {pais: {"pib_2000": np.random.uniform(1000, 40000),
                    "pib_2020": np.random.uniform(2000, 60000)}}
                for pais in países}

df = pd.DataFrame.from_dict(anos_dados, orient="index")
df["crescimento"] = (df["pib_2020"] / df["pib_2000"]) ** (1/20) - 1
df["crescimento"] *= 100 # em %
df["log_pib_2000"] = np.log(df["pib_2000"])

#  $\beta$ -convergência: crescimento  $\sim \beta * \log(\text{PIB inicial})$ 
X = sm.add_constant(df["log_pib_2000"])
y = df["crescimento"]

modelo = sm.OLS(y, X).fit()
print("TESTE DE  $\beta$ -CONVERGÊNCIA")
print("=" * 40)
print(modelo.summary())

# Coeficiente  $\beta$  negativo confirma convergência
beta = modelo.params["log_pib_2000"]
print(f"\n $\beta = \{beta:.4f\}$ ")
if beta < 0 and modelo.pvalues["log_pib_2000"] < 0.05:
    print("✅ Evidência de  $\beta$ -convergência (pobres crescem mais rápido)")
else:
    print("❌ Sem evidência de convergência")

# Gráfico
fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(df["log_pib_2000"], df["crescimento"], alpha=0.6, edgecolors="black")
ax.plot(df["log_pib_2000"], modelo.fittedvalues, color="red", linestyle="--")

ax.set_xlabel("Log PIB per capita (2000)")
ax.set_ylabel("Crescimento médio anual (%) 2000-2020")
ax.set_title(" $\beta$ -Convergência: Países Pobres Crescem Mais?")
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig("convergencia.png", dpi=150)

```

Projeto 3 — Curva de Kuznets Ambiental

Objetivo

Testar se a relação entre PIB per capita e emissões de CO₂ tem forma de U invertido.

```

import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

np.random.seed(42)
n = 100

# Simular dados com relação U-invertido
pib_per_capita = np.random.uniform(1000, 50000, n)
# emissões = termo_quadratICO com máximo no meio
emissao = 0.5 * pib_per_capita - 0.000006 * pib_per_capita**2
emissao += np.random.normal(0, 10, n) # ruído
emissao = np.maximum(emissao, 0) # não negativa

df = pd.DataFrame({"pib": pib_per_capita, "co2": emissao})
df["pib2"] = df["pib"] ** 2
df["log_pib"] = np.log(df["pib"])

# Modelo: CO2 = β0 + β1*PIB + β2*PIB2
X = sm.add_constant(df[["pib", "pib2"]])
y = df["co2"]

modelo = sm.OLS(y, X).fit()
print(modelo.summary())

b1 = modelo.params["pib"]
b2 = modelo.params["pib2"]

if b1 > 0 and b2 < 0:
    # Ponto de virada
    turning_point = -b1 / (2 * b2)
    print(f"\nPonto de virada do PIB per capita: ${turning_point:,.0f}")
    print("✅ Evidência de Curva de Kuznets Ambiental")
else:
    print("❌ Sem evidência de U-invertido")

# Gráfico
fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(df["pib"], df["co2"], alpha=0.5, edgecolors="black")

# Linha ajustada
x_range = np.linspace(df["pib"].min(), df["pib"].max(), 100)
y_pred = modelo.params["const"] + modelo.params["pib"] * x_range + \
    modelo.params["pib2"] * x_range**2
ax.plot(x_range, y_pred, color="red", linewidth=2, label="Ajuste quadrático")

ax.axvline(turning_point, color="gray", linestyle=":", label=f"Ponto virada: ${turning_point:,.0f}")
ax.legend()
ax.set_xlabel("PIB per capita (US$)")
ax.set_ylabel("Emissões de CO2 per capita")
ax.set_title("Curva de Kuznets Ambiental")
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig("kuznets.png", dpi=150)

```

Projeto 4 — Modelo de Previsão de Inflação

Objetivo

Criar um modelo SARIMA que prevê o IPCA para os próximos 12 meses.

```
from bcb import sgs
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller
import warnings
warnings.filterwarnings("ignore")

# 1. DADOS
ipca = sgs.get({"IPCA": 433}, start="2005-01-01")
serie = ipca["IPCA"]

# 2. TESTE DE ESTACIONARIEDADE
result = adfuller(serie.dropna())
print(f"ADF p-valor: {result[1]:.4f}")
if result[1] > 0.05:
    d = 1
    print("Série não estacionária → diferenciando (d=1)")
else:
    d = 0
    print("Série estacionária (d=0)")

# 3. SEPARAR TREINO/TESTE
train = serie[:-12]
test = serie[-12:]

# 4. SARIMA
# Ordem: (p, d, q) x (P, D, Q, s)
# s=12 para sazonalidade anual
modelo = SARIMAX(train,
                  order=(1, d, 1),
                  seasonal_order=(1, 1, 1, 12),
                  enforce_stationarity=False,
                  enforce_invertibility=False)

resultado = modelo.fit(dispatch=False)
print(resultado.summary())

# 5. PREVISÃO
forecast = resultado.forecast(steps=12)

# 6. AVALIAÇÃO
from sklearn.metrics import mean_absolute_error, mean_absolute_percentage_error

mae = mean_absolute_error(test, forecast)
mape = mean_absolute_percentage_error(test, forecast)

print(f"\n📊 AVALIAÇÃO DO MODELO")
print(f"Erro médio absoluto (MAE): {mae:.2f} pp")
```

```

print(f"Erro percentual (MAPE): {mape:.2f}%")

# 7. GRÁFICO
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(train.index[-60:], train[-60:], label="Histórico", linewidth=1.5)
ax.plot(test.index, test, label="Real", color="green", linewidth=1.5)
ax.plot(forecast.index, forecast, label="Previsão", color="red", linestyle="--",
        linewidth=1.5)
ax.fill_between(forecast.index,
                resultado.get_forecast(12).conf_int().iloc[:, 0],
                resultado.get_forecast(12).conf_int().iloc[:, 1],
                alpha=0.2, color="red", label="IC 95%")
ax.legend()
ax.set_title("Previsão IPCA - SARIMA(1,1,1)(1,1,1,12)")
ax.set_ylabel("IPCA mensal (%)")
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig("previsao_ipca.png", dpi=150)
plt.close()

print("\n📁 Gráfico salvo: previsao_ipca.png")
print("\nPrevisões:")
for data, valor in zip(forecast.index, forecast.values):
    print(f"    {data.date():} : {valor:.2f}%")

```

Projeto 5 — Regra de Taylor

Objetivo

Estimar a regra de Taylor para o Brasil: SELIC resposta à inflação e ao hiato do produto.

```

from bcb import sgs
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

# SELIC (11), IPCA acum 12m (13522), PIB (4380)
selic = sgs.get({"SELIC": 11}, start="2005-01-01")
ipca_acum = sgs.get({"IPCA_ACUM": 13522}, start="2005-01-01")
pib = sgs.get({"PIB": 4380}, start="2005-01-01")

df = pd.merge(selic, ipca_acum, on="Date")
df = pd.merge(df, pib, on="Date").dropna()
df.columns = ["selic", "ipca_acum", "pib"]

# Hiato do produto (desvio da tendência)
# Usando filtro HP
from statsmodels.tsa.filters.hp_filter import hpfilter
cycle, trend = hpfilter(df["pib"], lamb=14400)
df["hiato"] = cycle

# SELIC defasada (suavização)
df["selic_lag1"] = df["selic"].shift(1)

df = df.dropna()

# Regra de Taylor: SELIC =  $\beta_0$  +  $\beta_1$ *inflação +  $\beta_2$ *hiato

```

```

X = sm.add_constant(df[["ipca_acum", "hiato"]])
y = df["selic"]

modelo = sm.OLS(y, X).fit()
print("REGRAS DE TAYLOR - BRASIL")
print("=" * 40)
print(modelo.summary())

# SELIC prevista vs real
df["selic_prevista"] = modelo.fittedvalues

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(df.index, df["selic"], label="SELIC real", color="navy", linewidth=1.5)
ax.plot(df.index, df["selic_prevista"], label="SELIC prevista (Taylor)",
        color="crimson", linestyle="--", linewidth=1.5)
ax.legend()
ax.set_title("Regra de Taylor - Brasil")
ax.set_ylabel("SELIC (%)")
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig("regra_taylor.png", dpi=150)
plt.close()

print("\n📁 Gráfico salvo: regra_taylor.png")

```

Projeto 6 — Análise de Crise Cambial

Objetivo

Identificar períodos de crise cambial usando pressão no mercado de câmbio.

```

from bcb import sgs
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Câmbio (1) e Reservas (3546)
cambio = sgs.get({"CAMBIO": 1}, start="2000-01-01")
reservas = sgs.get({"RESERVAS": 3546}, start="2000-01-01")

df = pd.merge(cambio, reservas, on="Date").dropna()
df.columns = ["cambio", "reservas"]

# Variação cambial
df["cambio_var"] = df["cambio"].pct_change() * 100

# Índice de Pressão Cambial (EMP)
# Combina desvalorização cambial + perda de reservas
df["reservas_var"] = df["reservas"].pct_change() * 100
df["emp"] = df["cambio_var"] - df["reservas_var"] / df["reservas"].std()

# Identificar crises: EMP > 2 desvios padrão
threshold = df["emp"].mean() + 2 * df["emp"].std()
df["crise"] = df["emp"] > threshold

print(f"Períodos de crise cambial (EMP > {threshold:.1f}):")
crises = df[df["crise"]].index

```

```

for data in crises:
    print(f" {data.date()}")

# Gráfico
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(df.index, df["emp"], color="steelblue", linewidth=1)
ax.axhline(threshold, color="red", linestyle="--", label=f"Limiar crise
({threshold:.1f})")
ax.fill_between(df.index, threshold, df["emp"].max(),
                where=df["crise"], color="red", alpha=0.3, label="Crise")
ax.legend()
ax.set_title("Índice de Pressão Cambial - Brasil")
ax.set_ylabel("EMP (desvios padrão)")
ax.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig("crise_cambial.png", dpi=150)

```

Projeto 7 — TCC Estrutura Completa

Estrutura de um TCC em economia com Python:

```

meu_tcc/
├── dados/                    # Dados brutos (nunca modificados)
│   ├── bcb/                 # Baixados do BCB
│   ├── ipea/                # Baixados do Ipeadata
│   └── externos/            # Banco Mundial, FMI
├── scripts/                  # Código
│   ├── 01_baixar_dados.py
│   ├── 02_limpeza.py
│   ├── 03_analise_descritiva.py
│   ├── 04_modelos.py
│   └── 05_graficos.py
├── output/                   # Resultados
│   ├── tabelas/
│   ├── graficos/
│   └── relatorio/
├── environment.yml           # Ambiente replicável
└── README.md

```

Template de script de TCC

```

"""
TCC: [TÍTULO]
Autor: [SEU NOME]
Script: 04_modelos.py
Descrição: Estima modelos econométricos
"""

import pandas as pd
import numpy as np
import statsmodels.api as sm
from linearmodels.panel import PanelOLS
import warnings
warnings.filterwarnings("ignore")

# Configuração
np.random.seed(42)

```



```

SAVE_TABLES = True

# 1. CARREGAR DADOS
df = pd.read_csv("dados/painel_países.csv", decimal=",")
print(f"Dados carregados: {df.shape[0]} observações, {df.shape[1]} variáveis")

# 2. ESTATÍSTICAS DESCRITIVAS
desc = df.describe().round(3)
if SAVE_TABLES:
    desc.to_csv("output/tabelas/descritivas.csv", decimal=",")
print("\nEstatísticas descritivas salvas")

# 3. MODELO BASE
X = sm.add_constant(df[["inflacao", "educacao", "investimento"]])
y = df["crescimento_pib"]

modelo_base = sm.OLS(y, X).fit()
print("\nMODELO BASE (OLS)")
print("=" * 50)
print(modelo_base.summary())

# 4. MODELO COM EFEITOS FIXOS
df_panel = df.set_index(["pais", "ano"])
modelo_fe = PanelOLS.from_formula(
    "crescimento_pib ~ inflacao + educacao + investimento + EntityEffects",
    data=df_panel
).fit()

print("\nMODELO EFEITOS FIXOS")
print("=" * 50)
print(modelo_fe)

# 5. SALVAR RESULTADOS
# Coeficientes em CSV
coefs = pd.DataFrame({
    "variavel": modelo_fe.params.index,
    "coeficiente": modelo_fe.params.values,
    "p_valor": modelo_fe.pvalues.values,
    "erro_padrao": modelo_fe.std_errors.values
})
coefs.to_csv("output/tabelas/resultados_fe.csv", index=False, decimal=",")
print("\nResultados salvos em output/tabelas/")

print("\n✅ Modelos estimados com sucesso!")

```

Checklist para seu TCC

- ☐ Dados brutos salvos em dados/ (nunca modificá-los)
 - ☐ Scripts numerados na ordem de execução
 - ☐ Mesma seed (np.random.seed(42)) em todos os scripts
 - ☐ Tabelas exportadas para CSV com decimal=", "
 - ☐ Gráficos em PNG 300dpi
 - ☐ Ambiente replicável (environment.yml ou requirements.txt)
 - ☐ README explicando como reproduzir
-

Conclusão

Você tem agora 7 projetos completos que cobrem:

Projeto	Técnicas
Dashboard Macro	API BCB, pandas, matplotlib
β -Convergência	Regressão, scatter
Curva de Kuznets	Regressão quadrática
Previsão IPCA	SARIMA, séries temporais
Regra de Taylor	Econometria, filtro HP
Crise Cambial	Análise de indicadores
TCC	Estrutura completa

Pegue qualquer um, adapte com seus dados, e está pronto.